

Due October 6 at 3:00 pm

(6 questions, 125 points total)

1. (15 pts.) Boolean functions as propositional formulas

Write propositional formulas for the following Boolean functions:

- (a) The majority function $M(x_1, x_2, x_3)$, which is true if and only if a majority of its inputs are true.

Answer:

$$M(x_1, x_2, x_3) = (x_1 \wedge x_2) \vee (x_1 \wedge x_3) \vee (x_2 \wedge x_3) \quad (1)$$

- (b) The if-then-else function $ITE(i, t, e)$, which is equal to t if $i = \top$ and otherwise is equal to e (this also called a multiplexer; here we're looking at a simple version where the inputs t and e are a single bit).

Answer:

$$ITE(i, t, e) = (i \wedge t) \vee (\neg i \wedge e) \quad (2)$$

- (c) The threshold function $T_{\leq 5}(x_1, x_2, x_3, x_4)$, which is true if and only if the integer represented in binary as $x_1x_2x_3x_4$ is at most 5 (e.g. $T_{\leq 5}(1, 1, 0, 1) = \perp$ since $1101_2 = 13 \not\leq 5$).

Answer:

$$T_{\leq 5}(x_1, x_2, x_3, x_4) = \neg x_1 \wedge (\neg x_2 \wedge \neg x_3) \quad (3)$$

2. (20 pts.) Satisfiability and validity

For each of the following formulas, state whether it is satisfiable and/or valid.

- (a) $x \rightarrow \neg x$

Answer: simplifies to $x \vee \neg x$ which is a tautology and therefore satisfiable and valid.

- (b) $(x \vee y) \wedge (\neg x \vee y) \wedge (x \vee \neg y) \wedge (\neg x \vee \neg y)$

Answer: it is neither satisfiable nor valid. each of the 4 combinations of x, y always results in one box being false.

- (c) $(x \rightarrow y) \leftrightarrow (\neg x \vee y)$

Answer: this is definitionally true by the material conditional and is therefore satisfiable and valid.

- (d) $x \rightarrow (\neg x \rightarrow (y \wedge \neg y))$

Answer: $y \wedge \neg y$ is a contradiction and therefore false. $\neg x \rightarrow (y \wedge \neg y)$ true. the outermost material conditional is always true when the consequent is true. This is a tautology and therefore satisfiable and valid.

- (e) $(x \vee \neg x) \wedge (x \vee y \vee z)$

Answer: the first clause is always true. the second is sometimes true but not always so therefore satisfiable but not valid.

3. (20 pts.) Equivalence and equisatisfiability

For each of the following pairs of formulas, state whether they are equivalent and/or equisatisfiable.

- (a) x and y

Answer: they are equisatisfiable but not equivalent. they are only equivalent if x and y are the same variable.

- (b) $x \leftrightarrow y$ and $(x \rightarrow y) \wedge (\neg x \rightarrow \neg y)$

Answer: applying modus tollens on $\neg x \implies \neg y$ we see it is equivalent to $y \implies x$. so the second formula is equivalent to $x \implies y \wedge y \implies x$ which is the definition of $x \iff y$. therefore they are equivalent and equisatisfiable.

- (c) $(x \vee y \vee z) \wedge (x \vee \neg z)$ and $x \vee y$

Answer: they are not equivalent because z affects the result of the first while it does not in the second. they are equisatisfiable because if x is true, both formulas are satisfiable.

- (d) $x \rightarrow \neg x$ and $\perp$

Answer: the left formula is equivalent to $\perp$ so they are equivalent and equisatisfiable.

- (e) $(x \vee y) \wedge \neg y \wedge \neg x$ and $\neg(x \vee \neg x)$

Answer: they are both unsat and therefore equivalent but not equisatisfiable.

4. (20 pts.) Conversion to conjunctive normal form (CNF, a.k.a. product-of-sums)

Consider the formula $\neg((x \vee \neg y) \wedge z)$.

- (a) Find an equivalent formula in CNF without introducing new variables.

Answer:

$$\begin{aligned}\neg((x \vee \neg y) \wedge z) &= \neg(x \vee \neg y) \vee \neg z \\ &= \neg x \wedge (y \vee \neg z)\end{aligned}\tag{4}$$

- (b) Draw a circuit representing the original formula (not the one you created in part (a)).

Answer: see fig 1

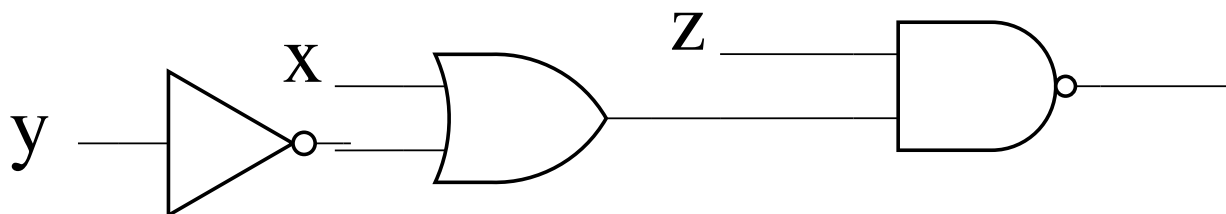


Figure 1:

- (c) Find an equisatisfiable formula in CNF using the Tseitin transformation.

Answer:

$$\begin{aligned}&c \wedge \\ &(a \vee y) \wedge (\neg a \vee \neg y) \\ &\wedge (\neg b \vee x \vee a) \wedge (b \vee \neg x) \wedge (b \vee \neg a) \\ &\wedge (\neg c \vee \neg b \vee \neg z) \wedge (c \vee b) \wedge (c \vee z)\end{aligned}\tag{5}$$

where c is the last wire, b is the middle wire and a is the neg y wire.

5. (40 pts.) Compactness of different encodings of a single function

$PAR_n(x_1, \dots, x_n) = x_1 \oplus \dots \oplus x_n$ is the parity function on n variables, true if and only if the number of true variables is odd (the symbol $\oplus$ is notation for the exclusive OR, a.k.a. XOR, operator). Let's look at how large different representations of this function are: for each of the types of representation below, give an encoding of PAR_n which is as small as possible¹ and compute its size asymptotically (e.g. $\Theta(n^2)$).

(Note: To write conjunctions or disjunctions of an arbitrary number of subformulas, you can use the notation:

$$\bigwedge_{i=1}^k \phi_i = \phi_1 \wedge \dots \wedge \phi_k \quad \bigvee_{i=1}^k \phi_i = \phi_1 \vee \dots \vee \phi_k$$

This is similar to the usual notation $\sum_{i=1}^k f(i)$ for sums. You can also write $\bigwedge_{s \in S} \phi_s$ if you have a set of subformulas indexed by objects in the set S instead of integers, and you can put constraints on indices in the same way you can when using sums, like $\sum_{\substack{1 \leq i, j \leq k \\ i \neq j}} f(i, j)$.

- (a) A DNF (disjunctive normal form, a.k.a. sum-of-products) formula.

Answer:

$$PAR_n(x_1, \dots, x_n) = \bigvee_{a \in S} \left(\bigwedge_{j \in n} \phi_{a_j}(x_j) \right) \quad (6)$$

where $S = \{a \in \{0, 1\}^n \mid \sum_{i \in n} a_i \pmod{2} = 1\}$ and where $\phi_i(x_j)$ evaluates to true iff $i = x_j$

- (b) A propositional formula which can only use the operators $\wedge$, $\vee$, and $\neg$ (this is sometimes called a “Boolean formula”).

(Hint: Try to define PAR_n recursively, in a way that your overall formula won't have exponential size.)

Answer:

let A compute the parity of $x_1, \dots, x_{\lfloor n/2 \rfloor}$ and B compute the right half, $x_{\lfloor n/2 \rfloor + 1}, \dots, x_n$

$$PAR_n = (A \wedge \neg B) \vee (\neg A \wedge B)$$

A, B are actually just recursive calls to $PAR_{\approx n/2}$. the recurrence is of size

$$S(n) = 4S\left(\frac{n}{2}\right) + O(1)$$

which is $O(n^2)$

- (c) An unrestricted propositional formula (i.e. you can use $\rightarrow$ and $\leftrightarrow$).

Answer: for odd n ,

$$x_1 \leftrightarrow x_2 \cdots \leftrightarrow x_n \quad (7)$$

for even n

$$\neg x_1 \leftrightarrow x_2 \cdots \leftrightarrow x_n \quad (8)$$

This works because $\leftrightarrow$ automatically tracks parity. when a 0 is found, it flips the sign and keeps that sign until it finds another 0. similar to 1.

- (d) A BDD (reduced and ordered; you may pick a variable order).

Answer: the bdd order is just 1 to n . the top layer has one node, x_1 , and each subsequent layer has 2. Odd and Even versions are different by swapping the first layer outputs.

see fig 2

¹You don't have to prove your encoding is (asymptotically) optimal, although it's not too hard to do this for (a), (c), and (d) — feel free to give it a try. On the other hand, the precise lower bound for (b) was a major result!

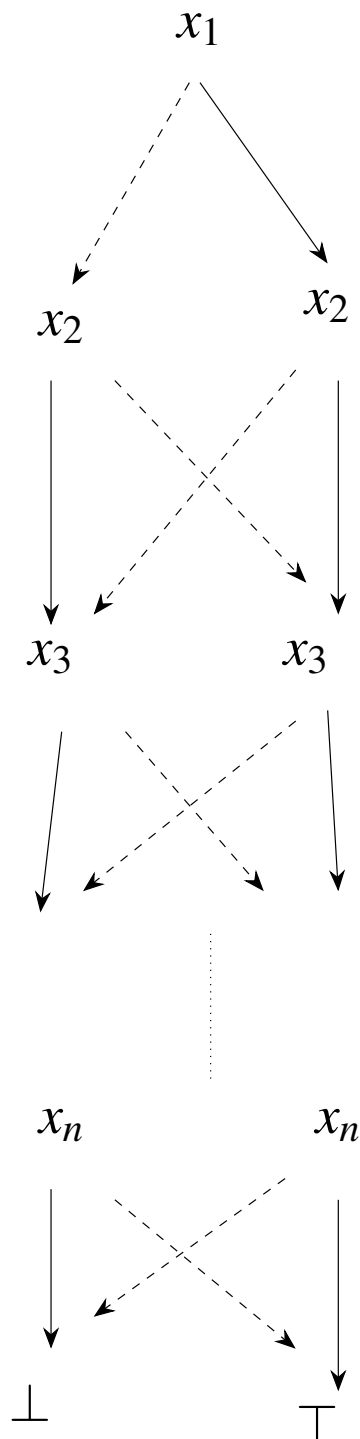


Figure 2: 5d. Odd version BDD. swap first layer arrows (x_1 to x_2) for even version

6. (10 pts.) Variable ordering in BDDs

Consider the formula $(x_1 \wedge x_2) \vee (x_3 \wedge x_4)$.

- (a) Construct an ROBDD with the variable order $x_1 > x_2 > x_3 > x_4$.

Answer: see fig 3

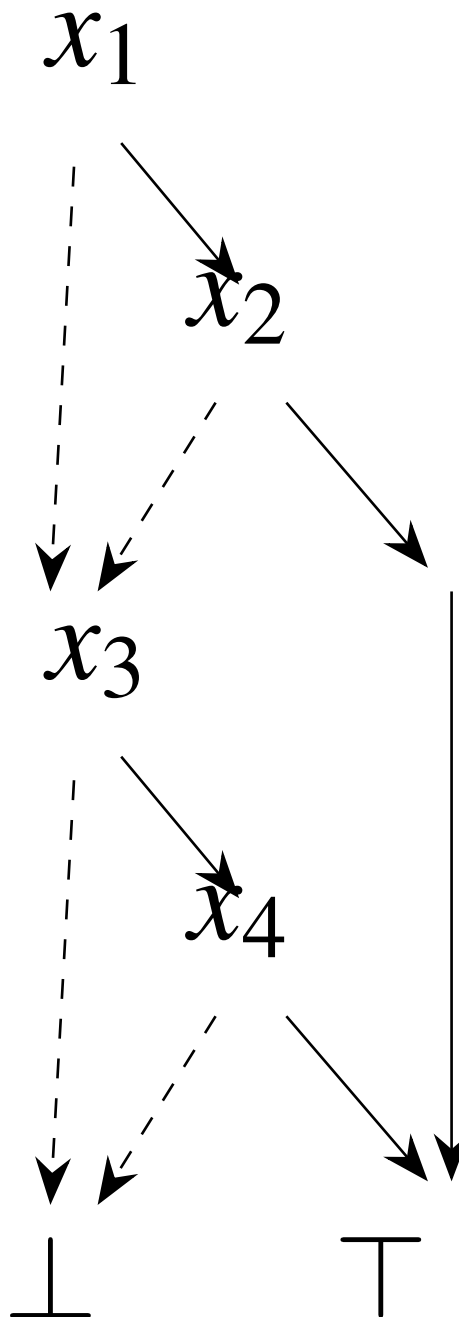


Figure 3: 6a

(b) Construct an ROBDD with the variable order $x_1 > x_3 > x_2 > x_4$.

Answer: see fig 4

N.B. If you generalize this formula to have n pairs of variables instead of just 2, the first variable order (increasing numbers) will yield a BDD of size $\Theta(n)$, while the second (odd variables first, then even variables) will yield a BDD of size $\Theta(2^n)$. So the choice of variable order really does make a difference in BDDs! If you're interested, you could try to prove these sizes.

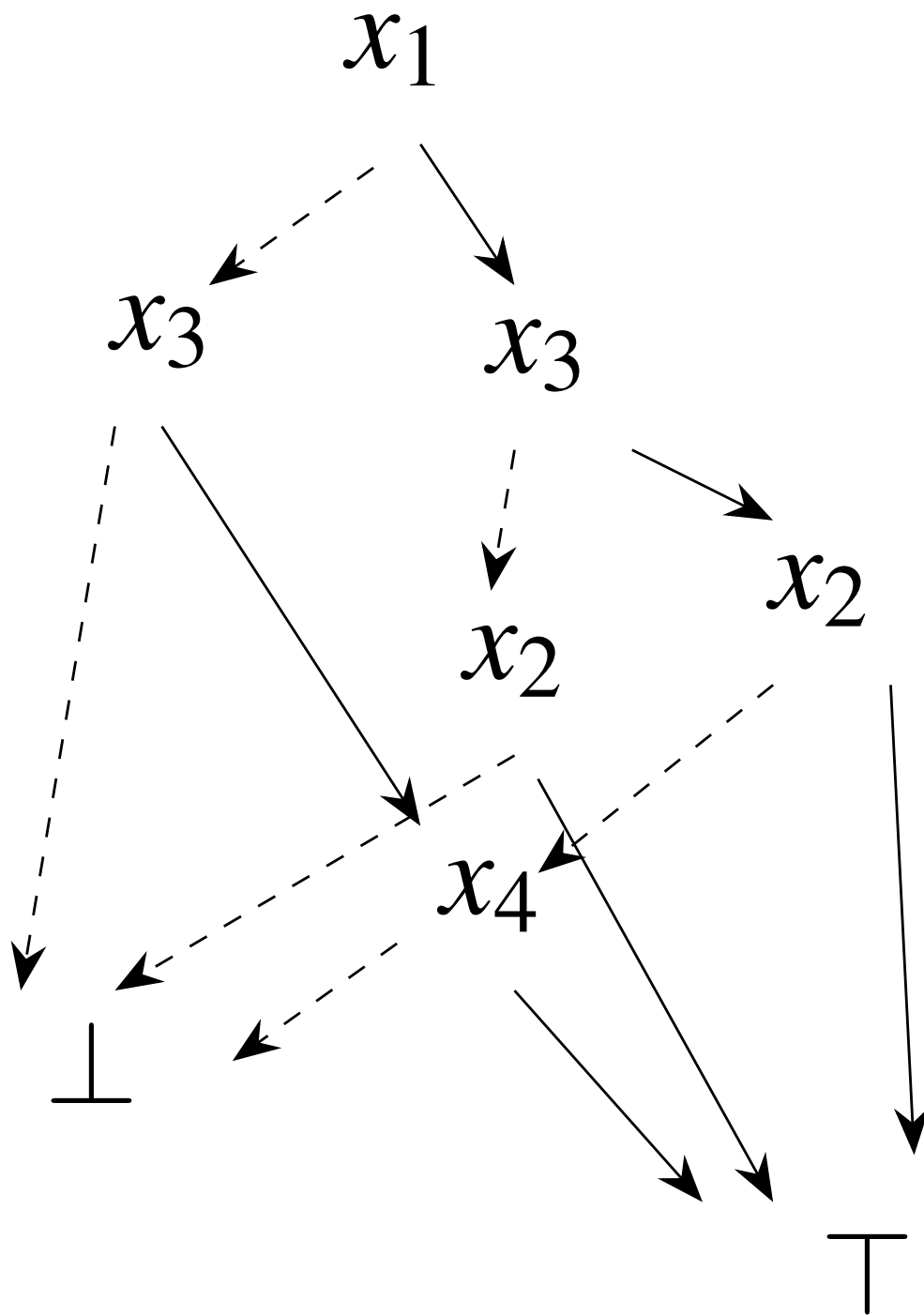


Figure 4: 6b